

Chains Instead of Slots

Lecture Notes — Week 6: Singly Linked Lists: Nodes, Links, and Pointer Surgery

Auqib Hamid Lone

Department of Computer Science & Engineering
Government College of Engineering & Technology (GCET), Ganderbal

September 26, 2026

6.1 Why This Week Exists: The Array Charges Rent

Read alongside: Horowitz & Sahni, *Fundamentals of Data Structures*, Ch. 4 (chapter-level); Aho, Hopcroft & Ullman, *Data Structures and Algorithms*, Ch. 2 (chapter-level).

Four questions organize this chapter, stated here exactly as the deck states them: why does inserting one name into a full array cost n moves; what does a node carry that an array slot does not; which linked shape fits which job — singly, doubly, or circular; and how do three pointers turn a chain around without copying it. Compressed to a plan: price the array’s weakness, build the node, tour the family, then operate — insert, delete, search, display — and finish with reverse.

The handoff from Week 5 is exact. Week 5 gave us the queue contract (**enqueue** at **rear**, **dequeue** from **front**), the circular fix — $\text{rear} = (\text{rear} + 1) \bmod n$ with the **count** convention (Lab P5) — dequeues, and array-based priority queues (Lab P6), then closed Unit I for Class Test 1. Every structure so far lives in *numbered slots*: an array owns its elements by position, and position is both its power and its tax. Its closing promise was “what changes when nodes carry their own links instead of sitting in numbered slots.” This week’s job frees the elements from their slots: each one carries the address of the next, and we price what we gain (cheap insert/delete) against what we lose (random access).

The calculator pipeline makes the need concrete. Weeks 3–5 built an expression processor: tokenize, check brackets, convert, evaluate, buffer jobs in arrival order — one program learning new tricks each week. The new deficiency: evaluating $(a+b)*c$ needs the *values* of **a**, **b**, **c** — identifiers arrive at unpredictable times, in unpredictable numbers, and a fixed array demands we guess the capacity up front. The same need appears everywhere: OS free-lists of memory blocks, browser back-forward chains, music playlists, and (Week 7) polynomials with missing terms. The course design principle fires again here: every structure in this course answers a deficiency the previous one just exhibited. The array’s deficiency is fixed capacity plus shifting — the linked list is the answer.

Two arc notes close the framing. Week 1 gave us **malloc/free** and **NULL**; Week 2 gave the pricing habit (operation + bound + case) — and every claim in this chapter repeats that shape. Week 7 re-implements the stack and the queue on top of *this* week’s chain, adds the backward link (**p->prev**), and spends the polynomial application; Week 10 replays the same symbol table as a hash table — $O(n)$ as a list vs. $O(1)$ expected hashed. Two new symbols today: **head** (first-node pointer, **NULL** when empty) and **p->next** (the successor link). The arc again: state the contract, implement it once, analyse what it costs.

6.2 From Slots to Chains: The Node and Its Price

Read alongside: Karumanchi, *Data Structures and Algorithms Made Easy*, Ch. 3, pp.74–162 (PDF) — node anatomy, the array-vs-list comparison, and the five core operations; Sedgewick & Wayne, *Algorithms*, Sec. 1.3, p. 142 — the “Linked lists” definition.

6.2.1 The slot tax

An array of n integers holds $A[0..n-1]$ back to back. Insert x at position k and every element from k to $n - 1$ must shift one slot right — up to n moves, $O(n)$ worst case. Delete at k and the same crowd shifts left. Growing past capacity means `malloc` a bigger block, copy all n , `free` the old one. **Position is the tax:** the array finds any element in $O(1)$ worst case by arithmetic, but pays linear time whenever the lineup itself changes. The question that buys the list: what if each element carried the address of its successor, so the lineup could change by rewriting one address instead of moving n elements?

Figure 6.1 draws the contrast the deck draws. Top: four array boxes 10, 20, 30, 40 sitting side by side at $A[0..3]$ — neighbours by arithmetic. Bottom: three heap boxes 10, 20, 30 ending in `NULL`, entered through `head` — neighbours by address, each box allowed to live anywhere in the heap.

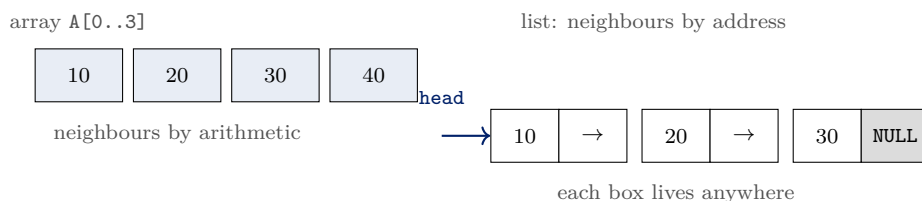


Figure 6.1: Array slots vs. linked nodes: neighbours by index arithmetic (top) against neighbours by address, followed from `head` to `NULL` (bottom).

6.2.2 The node: data plus one address

Definition (Linked list). A *linked list* is either empty ($\text{head} = \text{NULL}$) or a node holding data plus a reference to the rest of the list. In C: `struct node { int data; struct node *next; };` (Sedgewick Sec. 1.3, p. 142; Karumanchi Ch. 3, pp.74–162 PDF) [2, 1].

Each node is separately `malloc`’d — Week 1’s allocation trace, one box per element, scattered across the heap (stack grows down, heap up, same orientation as Week 1). `head` is the *only* entry point: lose it and the whole chain leaks. `p->next` of the last node is `NULL`. The pointer/dynamic-allocation concepts behind node creation are Wirth’s ground (general treatment); the C syntax above is the course Notation Registry’s, never Sedgewick’s Java nor Wirth’s Oberon.

Example 6.2.1 (Worked example: build 10–20–30 by hand). Start with `head = NULL`: empty, `head` points nowhere. Insert 10 at the front: `malloc` one box, fill it, point it at `NULL`, swing `head` over — heap reads `head` → [10/NULL]. Insert 20 at the front: fresh box, fill with 20, point its link at the old chain (the 10-box) first, then swing `head` — heap reads `head` → [20/·] → [10/NULL]. Insert 30 at the end: walk from `head` across the 20-link and the 10-link to the tail, append the fresh 30-box, terminate it with `NULL` — heap reads `head` → 20 → 10 → 30 → `NULL`. Read the arrows, not the addresses: the three boxes may sit at any heap locations — order lives in the links, never in the

layout. Every arrow written is one pointer assignment; every box is one *malloc* that a matching *free* must one day reclaim.

6.2.3 The honest bill

operation	array	singly linked list
access i -th element	$O(1)$ worst case	$O(n)$ worst case (walk i links)
insert/delete at front	$O(n)$ worst case (shift)	$O(1)$ worst case
insert/delete at end	$O(1)^*$ amortized	$O(n)$ worst case (no tail link)
memory	fixed, contiguous	grows one node at a time

The asterisk is load-bearing: append is $O(1)$ amortized only with spare capacity; a full array pays the copy-everything reallocation. There is no winner — only workloads: many positional reads $\rightarrow$ array; many front insertions and deletions $\rightarrow$ list. State the workload, then choose.

6.3 The Family and Where It Pays

Read alongside: Karumanchi, Ch. 3, pp.74–162 (PDF) — the list variants and the rule of thumb for reaching for a list (standard treatment for the naming).

6.3.1 Three shapes of chain

Definition (Singly, doubly, circular). A *singly* linked list carries one link per node and ends in *NULL* — cheapest per node, and today’s subject. A *doubly* linked list adds $p \rightarrow \text{prev}$ for backward walks at twice the link maintenance (Week 7). A *circular* list aims the last link home instead of at *NULL* — round-robin buffers and schedulers (Week 7).

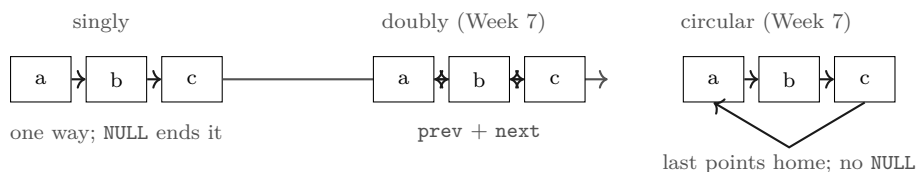


Figure 6.2: Three chain shapes: singly (one way, NULL-terminated), doubly (prev + next), circular (last link returns home).

6.3.2 Applications: where chains earn their keep

Definition (Rule of thumb). Reach for a list when the *number of items is unpredictable* and *insertions and deletions land anywhere* — the two places arrays charge the slot tax. (Karumanchi Ch. 3, pp.74–162 PDF; standard treatment) [1].

Three instances, exactly as the deck states them. **Symbol table** (our calculator): identifiers come and go; lookup walks the chain — $O(n)$ worst case, the bill Week 10’s hash table will cut to $O(1)$ expected. **OS memory management**: free blocks linked as they are freed and re-split on demand — no fixed table of blocks. **Polynomials** (Week 7 preview): $5x^{100} + 3x^2 + 1$ stores three nodes, not 101 array slots mostly holding zero.

Example 6.3.1 (Prediction check: 1 000 editor lines, 500 insertions at the top). *A text editor holds 1 000 lines and the user inserts one line at the top, 500 times. Array or singly linked list? Of A: array (indexing wins); B: linked list; C: identical either way — the answer is B. A confuses the workload: indexing only matters if we jump to line numbers, but here the workload is all front insertions. C ignores the table two sections back: price the workload, not the structure. Each list insert rewrites one **next** pointer — $O(1)$ worst case — where the array shifts up to n lines every time. Reads are sequential either way; the insertions decide.*

6.4 Pointer Surgery: Insert, Delete, Search, Display

Read alongside: Karumanchi, Ch. 3, pp.74–162 (PDF) — the pointer-surgery sequences (insert at front/position/end, delete by bypass-and-free) and the iterative reverse loop.

6.4.1 Insert at the front: two assignments

Pointer surgery (Karumanchi Ch. 3, pp.74–162 PDF):

```
newNode = malloc(sizeof(struct node)) // 1. fresh box
newNode->data = x                     // 2. fill it
newNode->next = head                  // 3. point at old chain FIRST
head = newNode                       // 4. THEN swing head over
```

Order is the whole lesson: step 4 before step 3 orphans the chain — **head**’s old value is gone with no copy anywhere. Cost: two pointer writes, no walk — $O(1)$ worst case. Always check `malloc` for `NULL` before dereferencing.

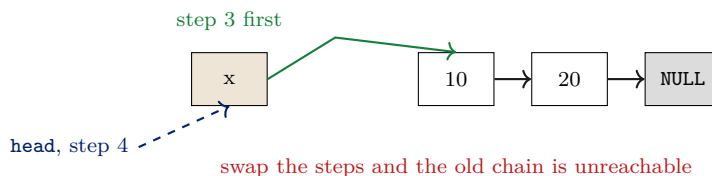


Figure 6.3: Front-insert surgery: the new box points at the old first node (step 3, solid) *before* **head** swings over (step 4, dashed). Either arrow alone is a broken list.

Trace it with a finger: new box points at the old first node, *then* **head** moves. Either arrow alone is a broken list.

Example 6.4.1 (Worked trace: front-insert 5 onto `head → 10 → 20 → NULL`). *malloc the fresh box. Fill: `newNode->data = 5`. Step 3: `newNode->next = head` — the new box now points at the 10-node while `head` still guards the old chain. Step 4: `head = newNode`. Final heap: `head → [5|.] → [10|.] → [20|NULL]`. Now the failure mode, step by step: swing `head` first and `head` points at the 5-box whose `next` field is still garbage; the address of the 10-node survives nowhere — two live nodes orphaned, heap still billing, exactly the leak Week 1 named.*

6.4.2 Insert at a position and at the end

The walk-then-splice pattern:

```
p = head; for (i = 0; i < pos-1; i++) p = p->next
newNode->next = p->next; p->next = newNode
```

Walk to the predecessor ($O(n)$ worst case), then the same two writes as front-insert in the same order — new node's link first, predecessor's link second. End-insert is position n : walk all n links — $O(n)$ worst case without a tail pointer. Two guards: inserting into an empty list sets **head** directly (there is no predecessor to walk to); walking past NULL is a bug, not an edge case — check $p \neq \text{NULL}$ as you walk.

6.4.3 Delete: find the predecessor, bypass, free

Delete the first occurrence of x :

```
if head->data == x: temp = head; head = head->next
else: find p with p->next->data == x
      temp = p->next; p->next = temp->next
free(temp)
```

Three acts: **bypass** (rewrite one link), **reclaim** (**free** exactly once), **never touch temp** after the free — that is Week 1's dangling pointer by name. Search-then-delete costs the search: $O(n)$ worst case; the bypass itself is $O(1)$ worst case once p is in hand.

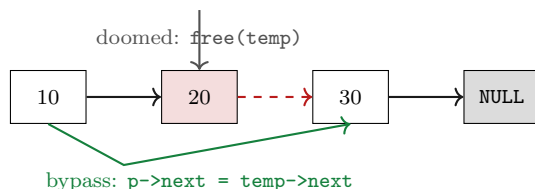


Figure 6.4: Deleting 20: node 10's link jumps over it straight to 30 (solid bypass), the old $20 \rightarrow 30$ arrow dies (dashed), then 20's box is freed.

Deleting 20, step by step: p sits on 10 with $p \rightarrow \text{next} \rightarrow \text{data} = 20$, so $\text{temp} = p \rightarrow \text{next}$ (the 20-box). Bypass: $p \rightarrow \text{next} = \text{temp} \rightarrow \text{next}$ — node 10's link jumps over 20 straight to 30, drawn solid. Reclaim: **free(temp)** — 20's box returns to the heap. Skip the free and the node still sits in the heap with nobody pointing at it — Week 1's leak, drawn. Touch **temp** afterward and it is Week 1's dangling pointer, drawn.

6.4.4 Search and display: the two walks

Both are one loop over $p = p \rightarrow \text{next}$:

```
search(x): p = head; while (p != NULL && p->data != x) p = p->next
           return (p != NULL) // found or walked off the end
display(): p = head; while (p != NULL) { print p->data; p = p->next }
```

Search is $O(n)$ worst case (the miss walks everything) and $O(1)$ best case (head holds x) — name the case, per the Week 2 habit. Display never modifies: the traversal pointer p moves while **head** stays put. Moving **head** itself loses the list.

Example 6.4.2 (Worked trace: search and display on $\text{head} \rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow \text{NULL}$). *Search for 20: $p = \text{head}$ ($10 \neq 20$, advance); p on 20 (match) — return found after 2 visits, and **head** never moved. Search for 99: visit 10, 20, 30, then $p = \text{NULL}$ — return not-found after walking everything, the $O(n)$ worst case exhibited. Display: p starts at **head**; print 10, advance; print 20, advance; print 30, advance; $p = \text{NULL}$, stop. Output 10 20 30; **head** still points at 10 throughout.*

6.5 Reverse, Costs, Close

Read alongside: Karumanchi, Ch. 3, pp.74–162 (PDF) — the iterative three-pointer reverse and the operation cost table.

6.5.1 Reverse: motivation before mechanism

The playlist just queued 500 songs oldest-first; the user taps “play newest first.” Copying all 500 into a second list doubles the memory for a rearrangement. The chain already holds every link we need — each arrow just points the wrong way. Reversal re-points n arrows using three pointers and $O(1)$ extra space. Socratic pause, exactly as the deck poses it: with only `prev`, `curr`, and `next`, which arrow must be saved *before* overwriting `curr->next` — and why? The answer (check your instinct): `next = curr->next` first — overwriting `curr->next` destroys the only path to the rest of the chain.

6.5.2 Iterative reverse: three pointers, one pass

The loop (Karumanchi Ch. 3, pp.74–162 PDF):

```
prev = NULL; curr = head
while (curr != NULL):
    next = curr->next // save the rest of the chain
    curr->next = prev // flip this arrow
    prev = curr      // advance both walkers
    curr = next
head = prev
```

Each node visited exactly once: $T(n) = n$ link flips — $O(n)$ worst case, $O(1)$ auxiliary space (three pointers, however long the list). Trace $10 \rightarrow 20 \rightarrow 30$ once on paper before the lab: after the second iteration the chain reads $20 \rightarrow 10 \rightarrow \text{NULL}$ with `curr` parked at 30 — the worked trace below shows every intermediate state.

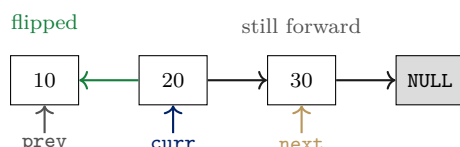


Figure 6.5: Mid-pass snapshot after one flip: 10’s arrow points back at NULL while $20 \rightarrow 30$ still runs forward; the flipped/forward frontier walks right until `curr = NULL`.

Example 6.5.1 (Worked trace: reverse $10 \rightarrow 20 \rightarrow 30 \rightarrow \text{NULL}$). *Setup:* `prev = NULL`, `curr = head` (the 10-node). *Iteration 1:* `next = curr->next` (save the 20-node); `curr->next = prev` (10 now points at NULL); `prev = curr` (10); `curr = next` (20). *State:* $10 \rightarrow \text{NULL}$ flipped, `curr` on 20 — Figure 6.5. *Iteration 2:* `next = 30`-node; 20’s arrow flips to 10; `prev` on 20; `curr` on 30. *State:* $20 \rightarrow 10 \rightarrow \text{NULL}$ with `curr` parked at 30, exactly as the deck promises. *Iteration 3:* `next = NULL`; 30’s arrow flips to 20; `prev` on 30; `curr = NULL` — loop exits. `head = prev` (the 30-node). *Final:* $head \rightarrow 30 \rightarrow 20 \rightarrow 10 \rightarrow \text{NULL}$. Three flips, one pass, no copy.

6.5.3 The full price list, mistakes, and summary

Singly linked list, Week 2 style — every bound names its operation and its case:

operation	bound	note
insert at front	$O(1)$ worst case	two writes; order matters
insert at end / position	$O(n)$ worst case	the walk dominates
delete (value known by walk)	$O(n)$ worst case	bypass itself $O(1)$
search	$O(n)$ worst / $O(1)$ best	miss walks everything
display	$\Theta(n)$	must visit all n
iterative reverse	$O(n)$ worst, $O(1)$ space	one pass, three pointers

“Insert is $O(1)$ ” without saying *where* is half an answer.

Three classic mistakes, stated exactly as the deck states them. **Rewriting head before saving it** — in front-insert (step order) and in reverse (unsaved **next**): the chain’s only entry address is gone, and the heap still bills you for every orphaned node. **Walking off the end** — $p = p \rightarrow \text{next}$ past **NULL** dereferences nothing: check $p \neq \text{NULL}$ in every loop condition, and test empty and single-node lists first. **Freeing and still pointing** — using **temp** after **free(temp)**, or forgetting the free entirely: Week 1’s dangling pointer and leak, now wearing list clothing. How to revise: re-trace the insert, delete, and reverse snapshots with a covered answer column — the exam is traces, not definitions.

In summary: **chain over slots** — unpredictable size plus anywhere-edits beat $O(1)$ indexing; that is when the list wins, otherwise the array keeps its crown. **Node** — `struct node { int data; struct node *next; };` with **head** as sole entry and **NULL** as terminator; singly, doubly, and circular shapes for different jobs. **Five operations** — front-insert $O(1)$ worst case; end/position insert, delete, and search $O(n)$ worst case; display $\Theta(n)$; three-pointer reverse $O(n)$ worst case in $O(1)$ space. Next week: the backward link ($p \rightarrow \text{prev}$), circular lists, the stack and queue reborn on chains — and polynomials that only store what is nonzero.

Example 6.5.2 (Before-you-go predictions, with the deck’s answers). *Answer from the algorithms, then check in the lab. (1) List $\text{head} \rightarrow 5 \rightarrow 15 \rightarrow \text{NULL}$: insert 10 at the front — $\text{newNode} \rightarrow \text{next} = \text{head}$, then $\text{head} = \text{newNode}$; the order is the answer, and the heap ends $\text{head} \rightarrow 10 \rightarrow 5 \rightarrow 15 \rightarrow \text{NULL}$. (2) Delete 15 above: rewrite 5’s **next** to **NULL** (bypass 15), then free 15’s node; skipping the free leaks exactly one box. (3) Reverse $\text{head} \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow \text{NULL}$: after the loop $\text{head} \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow \text{NULL}$ at $O(n)$ worst case, $O(1)$ space — three flips, three pointers, no copy.*

Exercises

Answer from the algorithms, then check against the Solutions block: pointer surgery is Lab work next week, but every trace below runs on paper first.

1. Front-insert trace: start from **head** = **NULL**. Insert 10, then 20, then 30 — all at the front. Give the heap chain after each step, and state which assignment order keeps the chain alive.
2. Position insert: on $\text{head} \rightarrow 20 \rightarrow 10 \rightarrow 30 \rightarrow \text{NULL}$, insert 25 at position 2 (0-based). Which node does **p** stop on, what are the two splice assignments in order, and what is the worst-case bound?
3. Delete trace: on $\text{head} \rightarrow 10 \rightarrow 20 \rightarrow 30 \rightarrow \text{NULL}$, delete the first occurrence of 10 (the head case) and then delete 30 (the tail case). Write the bypass assignment and the freed node for each, and state the cost.
4. Search and display: on $\text{head} \rightarrow 5 \rightarrow 15 \rightarrow 25 \rightarrow \text{NULL}$, how many nodes does **search(15)** visit, how many does **search(99)** visit, and what does **display()** print? Name the case for each bound. (Genuinely new instance reusing the deck’s two-walk pattern.)

- Reverse trace: reverse `head` $\rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow \text{NULL}$ with the three-pointer loop. Give `prev/curr/next` after each iteration, the final `head`, and the time/space bounds.

Solutions

- After 10:** `head` $\rightarrow [10|\text{NULL}]$; **after 20:** `head` $\rightarrow 20 \rightarrow 10 \rightarrow \text{NULL}$; **after 30:** `head` $\rightarrow 30 \rightarrow 20 \rightarrow 10 \rightarrow \text{NULL}$. **Safe order:** `newNode->next = head` first, `head = newNode` second. Each step mallocs one box and fills it, links it at the old chain before swinging `head`. Reversed order orphans the old chain: `head`'s old value survives nowhere.
- p stops on the node holding 10 (index 1, the predecessor); splice** `newNode->next = p->next` **then** `p->next = newNode`; $O(n)$ worst case. Walk: `p = head` (20), advance once (10) — one step for `pos = 2`. Splice in deck order or the tail ($10 \rightarrow 30$) is lost. Final: `head` $\rightarrow 20 \rightarrow 10 \rightarrow 25 \rightarrow 30 \rightarrow \text{NULL}$. Bound: the walk dominates ($O(n)$ worst case); the two writes are $O(1)$.
- Delete 10 (head):** `temp = head`; `head = head->next` (the 20-node); `free(temp)` frees the 10-box. **Delete 30 (tail):** `p` stops on 20; `temp = p->next` (30); `p->next = temp->next` ($= \text{NULL}$); `free(temp)` frees the 30-box. $O(n)$ worst case each via the search; each bypass $O(1)$. Final chain: `head` $\rightarrow 20 \rightarrow \text{NULL}$. Forgetting either free leaks one box; touching `temp` after is the dangling pointer.
- search(15) visits 2 nodes** ($O(1)$ -side: second position, constant here but $O(n)$ worst case in general); **search(99) visits all 3 then NULL** ($O(n)$ worst case — the miss walks everything); **display()** prints 5 15 25 and leaves `head` on 5. Name operation + bound + case: “search is $O(n)$ worst case (miss), $O(1)$ best case (head hit).” Display is $\Theta(n)$ — it must visit all n .
- After iter 1:** `prev` on 1, `curr` on 2, `next` saved 2 (chain $1 \rightarrow \text{NULL}$). **After iter 2:** `prev` on 2, `curr` on 3, `next` saved 3 (chain $2 \rightarrow 1 \rightarrow \text{NULL}$). **After iter 3:** `prev` on 3, `curr` = `NULL`, `next` = `NULL` (chain $3 \rightarrow 2 \rightarrow 1 \rightarrow \text{NULL}$). **Final head = prev** (the 3-node). $O(n)$ worst case time, $O(1)$ auxiliary space. Saving `next` before each flip is load-bearing: without it the rest of the chain is unreachable.

References

- [1] Narasimha Karumanchi. *Data Structures And Algorithms Made Easy*. CareerMonk Publications, 2017. This calibre-generated edition carries no printed folio numbers; page numbers cited in CS301 material are PDF page indices. NOT a source for Week 1’s C-specific pointer/malloc content — see this course’s supporting-books index for the coverage check.
- [2] Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley, 4th edition, 2011.